

인터뷰 키트 · 주문·재고 API v2

제출 [@ 4c595e36efb5489dd0234d834cfdb583048ccfc1](https://github.com/wanted-hack-inchyangv/taeyun-order-api)

기준 버전 v2-be07fb44

워크벤치 <https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?tab=questions>

질문 11개 · 필수 3개 · 권장 4개 · 선택 4개

이력서 연결 질문에는 지원자 이력서의 인용이 들어 있습니다. 복사한 내용은 면접 담당자에게만 공유해 주세요.

45분 진행안 구간 합 44분

도입 5분	
실패 디브리핑 16분	Q1 필수 · 실패 디브리핑 · 8분 Q2 필수 · 실패 디브리핑 · 8분
설계 트레이드오프 6분	Q4 권장 · 설계 트레이드오프 · 6분
강점 확인 12분	Q3 필수 · 강점 확인 · 6분 Q5 권장 · 강점 확인 · 6분
지원자 질문 5분	

60분 진행안 구간 합 55분

도입 5분	
실패 디브리핑 16분	Q1 필수 · 실패 디브리핑 · 8분 Q2 필수 · 실패 디브리핑 · 8분
설계 트레이드오프 6분	Q4 권장 · 설계 트레이드오프 · 6분
강점 확인 12분	Q3 필수 · 강점 확인 · 6분 Q5 권장 · 강점 확인 · 6분
요구사항 확장 6분	Q6 권장 · 요구사항 확장 · 6분
이력서 연결 5분	Q7 권장 · 이력서 연결 · 5분
지원자 질문 5분	

Q1 필수 실패 디브리핑 원인 분석 · 8분

R-06 실행 기록에서 같은 키로 다른 본문을 보냈을 때 201이 돌아왔습니다. 요청이 들어온 뒤 저장된 기록과 본문을 비교하는 지점을 코드 어디에 두셨나요?

의도 먹등 키 충돌을 걸러내는 판단이 코드 어디에서 일어나는지 지원자가 스스로 짚는지 확인한다.

꼬리 질문

1. 저장된 기록과 새 본문이 다르다는 것을 어떤 값으로 판단하시나요?
2. 그 비교가 실행되지 않는 경로는 무엇인가요?
3. 비교 시점을 저장 전후 어디에 둘지 정할 때 어떤 기준을 쓰시겠어요?

좋은 답변의 신호

- 먹등 기록 조회와 본문 비교가 일어나는 위치를 코드에서 짚는다.
- 충돌 판정과 새 주문 생성이 갈라지는 조건을 구분해 설명한다.

우려 신호

- 201이 나온 이유를 도구나 하네스 문제로 돌린다.
- 기록 조회와 본문 비교 중 어느 쪽이 빠졌는지 말하지 못한다.

근거

R-06 · 먹등 키 충돌

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-06>

R-06 재생

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-06&run=8cdc4c86-574b-4883-a53e-e38c5c73fbac>

src/routes/order-routes.ts:25-28

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>

src/routes/order-routes.ts:20-23

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>

src/routes/order-routes.ts:30-33

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L30-L33>

R-07 실행 기록에서 같은 키의 동시 요청 10건 중 5건만 201이고 응답 id가 서로 달랐습니다. 같은 키의 요청이 동시에 들어올 때 주문 생성이 어떤 순서로 진행되는지 설명해 주시겠어요?

의도 동시 요청에서 역등성 기록과 재고 차감이 어떤 순서로 일어나는지 지원자가 설명하는지 확인한다.

꼬리 질문

1. 기록을 확인하는 시점과 기록을 남기는 시점 사이에 다른 요청이 끼어들면 어떤 일이 생기나요?
2. 재고가 0이 된 결과는 어느 단계에서 발생했다고 보시나요?
3. 같은 키의 요청을 한 번만 처리하도록 만들려면 어느 지점을 먼저 바꾸시겠어요?

좋은 답변의 신호

- 기록 확인과 기록 저장 사이의 간격을 문제 지점으로 짚는다.
- 동시 요청에서 재고 차감이 여러 번 일어나는 경로를 설명한다.

우려 신호

- 동시성 문제를 언급하지 않고 순차 호출만 가정한다.
- 재고가 0이 된 이유를 설명하지 못한다.

근거

R-07 · 같은 키 동시 요청

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-07>

R-07 재생

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-07&run=8ea0a2f4-fa28-4bca-9a29-a5d69153d088>

src/routes/order-routes.ts:25-28

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>

src/routes/order-routes.ts:20-23

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>

Q3 필수 강점 확인 경계 조건과 실패 처리 · 6분

R-03은 재고 부족 요청을 409로 거절하고 재고를 그대로 두었습니다. 재고와 요청 수량을 비교하는 판단을 어디에 두셨나요?

의도 통과한 거절 동작의 판단 위치와 기준을 지원자가 스스로 설명하는지 확인한다.

꼬리 질문

1. 재고와 같은 수량의 요청을 허용으로 본 기준은 무엇인가요?
2. 거절로 끝난 요청이 재고를 건드리지 않도록 어떤 순서를 쓰셨나요?
3. 이 판단을 다른 계층으로 옮기면 무엇이 달라지나요?

좋은 답변의 신호

- 비교 조건과 차감 순서를 구분해 설명한다.
- 재고와 같은 수량을 허용한 기준을 수치로 말한다.

우려 신호

- 통과 결과만 말하고 판단 위치를 짚지 못한다.
- 거절과 차감의 순서를 뒤바꿔 설명한다.

근거

R-03 · 재고 부족 거절

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-03>

R-03 재생

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-03&run=a9f2d522-edac-4707-ba44-aaf462541f61>

src/routes/order-routes.ts:25-28

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>

src/routes/order-routes.ts:20-23

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>

Q4 권장 설계 트레이드오프 설계와 변경 용이성 · 6분

가장 긴 함수가 라우팅 파일의 `createOrderRoutes`로 34줄입니다. 이 함수 안에서 HTTP 처리와 도메인 규칙이 어떻게 나뉘어 있는지 설명해 주시겠어요?

의도 라우팅 함수 안에서 HTTP 처리와 도메인 규칙의 경계를 지원자가 어떻게 인식하는지 확인한다.

꼬리 질문

1. 이 함수에서 요청·응답 객체에 직접 닿는 부분은 어디까지인가요?
2. 재고 차감과 역등성 판단은 어느 모듈에 있나요?
3. 이 함수를 나눈다면 어떤 경계를 먼저 그으시겠어요?

좋은 답변의 신호

- 함수 안에서 HTTP 의존 부분과 규칙 부분을 구분해 짚는다.
- 도메인 규칙이 요청·응답 객체에 닿는지 여부를 스스로 판단한다.

우려 신호

- 함수 길이를 문제로 보지 않고 그대로 두겠다고만 말한다.
- HTTP 처리와 도메인 규칙을 같은 것으로 본다.

근거

`src/routes/order-routes.ts:8-41`

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L8-L41>

R-12 · 설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-12>

R-04는 여러 잘못된 입력을 400으로 거절하고 재고를 그대로 두었습니다. 입력 검증을 통과한 요청만 역등성 기록에 남도록 어떤 순서를 쓰셨나요?

의도 검증과 역등성 기록의 순서를 지원자가 설명하는지 확인한다.

꼬리 질문

1. 검증 실패로 끝난 요청의 키가 기록에 남지 않는 이유는 무엇인가요?
2. 검증 항목을 추가할 때 수정할 위치는 어디인가요?
3. 검증을 도메인 규칙 안으로 옮기면 무엇이 달라지나요?

좋은 답변의 신호

- 검증과 기록 저장의 순서를 단계로 나눠 설명한다.
- 검증 실패 시 재고가 변하지 않는 이유를 순서로 설명한다.

우려 신호

- 검증과 기록의 순서를 구분하지 않는다.
- 검증 항목이 어디에 모여 있는지 짚지 못한다.

근거

R-04 · 입력 검증

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-04>

R-04 재생

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-04&run=4c931fe0-c90f-433c-b27c-42aae2620f8e>

src/routes/order-routes.ts:25-28

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>

src/routes/order-routes.ts:20-23

<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>

같은 서비스를 여러 인스턴스로 띄워 요청이 서로 다른 인스턴스로 들어가는 상황을 가정해 보겠습니다. 지금 구조에서 가장 먼저 바꾸실 부분은 어디인가요?

의도 인스턴스가 늘어날 때 어떤 상태 공유 지점을 먼저 손대는지 확인한다.

꼬리 질문

1. 그 부분을 먼저 고르신 이유는 무엇인가요?
2. 인스턴스마다 상태가 따로 있으면 같은 키의 요청은 어떻게 되나요?
3. 공유 저장소를 두는 대신 다른 방법을 쓴다면 무엇을 고려하시겠어요?

좋은 답변의 신호

- 인스턴스 간에 공유되지 않는 상태를 구체적으로 짚는다.
- 같은 키의 요청이 다른 인스턴스로 갈 때 생기는 문제를 설명한다.

우려 신호

- 인스턴스를 늘리는 것만 언급하고 상태 공유를 말하지 않는다.
- 도구 이름만 대고 바꿀 지점을 짚지 못한다.

근거

R-01 · 정상 주문 생성

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-01>

R-05 · 맥등 재전송

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-05>

R-06 · 맥등 키 충돌

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-06>

결제 웹훅 먹등 처리에서 같은 키로 다른 내용이 재전송되면 어떻게 처리하도록 설계하셨나요?

이력서: 결제 웹훅(payment webhook) 수신을 맡아 같은 웹훅이 재전송돼도 결제가 중복 반영되지 않도록 먹등(idempotent) 처리를 담당했습니다.

의도 이력서에 적힌 웹훅 먹등 처리의 충돌·동시성 조건까지의 보장 범위를 확인한다.

꼬리 질문

1. 그 서비스에서 먹등 키는 어떤 값으로 만드셨나요?
2. 같은 키의 요청이 동시에 두 번 들어오면 어느 단계에서 하나로 정리되나요?
3. 이번 과제의 인메모리 구현과 비교하면 저장 위치와 트랜잭션 경계에서 무엇이 달라지나요?

좋은 답변의 신호

- 키 저장 위치와 만료, 본문 비교 기준을 구체적으로 설명한다
- 동시 요청을 막는 지점(유일 제약·잠금)과 재시도 조건을 구분해 설명한다

우려 신호

- 먹등성을 '같은 요청은 한 번만 처리한다'는 일반론으로만 설명한다
- 같은 키에 다른 본문이 오는 경우를 다루지 않는다

근거

이력서 연결

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?tab=resume#claim-8bfc46b-3340-49eb-af96-aeac904aec3d>

R-05·먹등 재전송

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-05>

선택 질문 4개 · 22분

R-12 설계 항목에서 먹등성 기록의 저장과 조회가 어디에 있는지 보겠습니다. 인메모리 구현을 다른 저장소로 바꿀 때 주문 생성 로직에서 고치지 않아도 되는 경계를 어떻게 그으셨나요?

의도 먹등성 저장소를 교체할 때의 경계가 코드에서 어떻게 잡혀 있는지 확인한다.

꼬리 질문

1. 저장소를 바꿀 때 주문 생성 로직이 의존하는 것은 무엇인가요?
2. 새 오류 코드를 추가할 때 수정할 위치는 몇 곳인가요?
3. 상태 코드와 오류 코드 매핑은 어디에 모여 있나요?

좋은 답변의 신호

- 저장소 교체 시 영향 범위를 코드 위치로 말한다.
- 오류 코드 매핑이 모인 위치를 짚는다.

우려 신호

- 계층 분리를 일반론으로만 설명한다.
- 저장소 교체 시 고칠 위치를 짚지 못한다.

근거

R-12·설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-12>

Q9 선택 요구사항 확장 트레이드오프 판단과 확장 · 6분

프로세스를 다시 시작해도 주문과 상태가 남아야 하는 상황을 가정해 보겠습니다. 지금 구조에서 가장 먼저 바꾸실 부분은 어디인가요?

의도 영속성이 필요해질 때 어떤 상태 저장 지점을 먼저 손대는지 확인한다.

꼬리 질문

1. 그 부분을 먼저 고르신 이유는 무엇인가요?
2. 저장 위치를 바꾸면 기존 동작 중 무엇을 다시 확인하시겠어요?
3. 초기화 경로는 새 저장 구조에서 어떻게 동작해야 한다고 보시나요?

좋은 답변의 신호

- 메모리에만 있는 상태를 구체적인 위치로 짚는다.
- 저장 위치를 바꾼 뒤 재확인할 동작을 든다.

우려 신호

- 도구 이름만 대고 바꿀 지점을 짚지 못한다.
- 초기화 경로와의 관계를 말하지 않는다.

근거

R-01 · 정상 주문 생성

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-01>

R-02 · 상품·주문 조회

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-02>

R-05 · 맥등 재전송

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-05>

Q10 선택 이력서 연결 설계와 변경 용이성 · 5분

회사 예약 API에서 중복 예약 검증은 어느 계층에서 어떤 조건으로 수행하셨나요?

이력서: 독서 모임 예약(booking) API를 개발하며 같은 시간대에 같은 장소·모임이 중복 예약되지 않도록 검증 로직을 구현했습니다.

의도 이력서의 중복 예약 검증이 어느 계층에 위치하고 어떤 경계 조건을 다루는지 확인한다.

꼬리 질문

1. 겹침 판정은 시작·종료 시각을 어떤 비교식으로 처리하나요?
2. 검증과 저장 사이에 다른 요청이 끼어들면 어떻게 되나요?
3. 개인 프로젝트의 인메모리 저장 방식과 비교하면 무엇을 포기하게 되나요?

좋은 답변의 신호

- 겹침 조건의 경계(끝과 시작이 같은 경우)를 명시해 설명한다
- 검증 위치와 저장 시점의 순서를 근거와 함께 설명한다

우려 신호

- 겹침 판정을 '시간이 겹치면 거절'로만 설명한다
- 검증과 저장 사이의 경합 가능성을 언급하지 않는다

근거

이력서 연결

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?tab=resume#claim-70c6e48e-e85d-4bde-804c-633962c78ccd>

R-08 · 다른 키 동시 요청과 재고 하한

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-08>

계층을 분리한 뒤 신규 기능을 추가할 때 변경 범위가 실제로 어떻게 달라졌나요?

이력서: 모임·예약·결제 도메인의 라우팅(routes)·서비스(services)·저장소(repository) 계층을 분리하는 리팩터링에 참여해 신규 기능 추가 시 변경 범위를 줄였습니다.

의도 이력서의 계층 분리 리팩터링이 변경 범위에 미친 영향을 구체적 사례로 확인한다.

꼬리 질문

1. 그때 분리 기준은 무엇이었나요?
2. 계층 간 의존 방향을 어떻게 강제하셨나요?
3. 분리를 하지 않았을 때와 비교해 어떤 비용이 생겼나요?

좋은 답변의 신호

- 특정 기능 추가 시 수정한 파일과 계층을 구체적으로 제시한다
- 의존 방향과 테스트 대체 지점을 함께 설명한다

우려 신호

- '계층을 나눴다'는 수준에서 그친다
- 변경 범위가 줄었다는 근거를 제시하지 못한다

근거

이력서 연결

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?tab=resume#claim-d6b041a6-11b2-41d8-bc32-6df76acbc4ec>

R-12 · 설계·변경 용이성

<https://ohmyti.vercel.app/evaluations/f9d4d72a-72d0-4a2f-9907-11563827bcf4?criterion=R-12>

평가 척도 (역량별 4단계)

면접관이 기입할 때 쓰는 기준이며 시스템은 값을 채우지 않습니다.

경계 조건과 실패 처리

잘못된 입력과 자원 부족 같은 실패 경로를 예측하고 일관되게 처리한다.

- | | |
|----------------|---|
| 1 미흡 | 잘못된 입력이나 자원 부족 같은 실패 경로를 떠올리지 못한다. |
| 2 보완 필요 | 실패 경로를 묻으면 떠올리지만, 처리 방식과 응답 형식을 구체적으로 말하지 못한다. |
| 3 충족 | 경계값과 실패 경로를 스스로 나열하고, 각각의 응답 코드와 상태 변화를 설명한다. |
| 4 탁월 | 재시도 중 실패나 부분 실패처럼 실패가 겹치는 상황까지 다루고, 상태가 일관되게 남는 근거를 설명한다. |

동시성과 데이터 정합성

동시에 들어오는 요청에서도 데이터가 어긋나지 않도록 보호한다.

- | | |
|----------------|---|
| 1 미흡 | 동시에 들어온 요청 때문에 상태가 어긋날 수 있다는 점을 인식하지 못한다. |
| 2 보완 필요 | 경쟁 조건이 생길 수 있다는 점은 알지만, 어느 구간에서 생기는지 짚지 못한다. |
| 3 충족 | 경쟁이 생기는 구간과 그 구간을 보호하는 방법(락, 원자적 연산, 유일성 제약)을 코드 위치와 함께 설명한다. |
| 4 탁월 | 저장소가 바뀌거나 인스턴스가 늘어날 때 보호 방법이 깨지는 조건을 짚고, 대안을 비교해 설명한다. |

설계와 변경 용이성

책임을 나누어 요구가 바뀌었을 때 고칠 곳이 좁고 분명한 구조를 만든다.

- 1 미흡 코드 구조를 그렇게 나눈 이유를 설명하지 못한다.
- 2 보완 필요 구조는 설명하지만, 요구가 바뀌면 어디를 고쳐야 하는지 분명히 말하지 못한다.
- 3 충족 책임을 나눈 기준을 설명하고, 요구 변경 하나에 대해 고칠 위치를 코드에서 짚는다.
- 4 탁월 현재 구조의 한계를 스스로 짚고, 다른 구조와 비교해 선택에 따르는 비용을 설명한다.

원인 분석 면접에서만 확인

관측된 실패에서 근거를 따라 원인 코드까지 범위를 좁힌다.

- 1 미흡 재생 기록을 보고도 실패 원인을 추정하지 못한다.
- 2 보완 필요 원인 후보를 말하지만, 근거를 대지 못하고 추측에 머문다.
- 3 충족 재생 기록의 기대값·실제값에서 원인 코드 위치까지 스스로 좁힌다.
- 4 탁월 원인을 좁힌 뒤, 같은 원인으로 생길 수 있는 다른 결함과 재발을 막는 방법까지 제시한다.

트레이드오프 판단과 확장 면접에서만 확인

선택지의 비용과 이점을 기준을 세워 비교하고 조건이 바뀌면 판단을 조정한다.

- 1 미흡 선택지 사이의 차이를 설명하지 못하고 한 가지 방법만 제시한다.
- 2 보완 필요 선택지를 나열하지만, 무엇을 기준으로 골라야 하는지 말하지 못한다.
- 3 충족 정합성, 성능, 복잡도 같은 판단 기준을 밝히고, 과제 조건에서 그 방법을 고른 이유를 설명한다.
- 4 탁월 트래픽 증가나 다중 인스턴스처럼 조건이 바뀌면 선택이 어떻게 달라지는지 근거와 함께 설명한다.